

Action-Bound Injective Authorization under Signer Harvesting

Iman Schrock
EMILIA Protocol, Inc.
team@emiliaprotocol.ai
ORCID: 0009-0004-0290-5433

Preprint. August 2026.

Abstract

A valid signature proves that a key signed a message. It does not, by itself, prove that an authorization collector bound the signature to the action later admitted, that two collected signatures belong to one approval ceremony, or that the same authorization was admitted at most once. These gaps matter when the collector renders requests, solicits signatures, and retains the audit record, and is itself adversarial. We define Action-Bound Injective Authorization (ABIA), a protocol property for this signer-harvesting setting. An approver signs a closed encoding of an exact action and an independently issued authorization instance. An enforcement domain admits the authorization only through a linearizable consume-if-unused transition. We give explicit transplantation, duplicate-admission, and quorum-substitution games. We prove that a successful transplantation or quorum substitution yields a signature forgery or hash collision, while duplicate admission yields an atomicity failure or enforcement bypass. The result deliberately separates cryptographic authenticity from the state-service assumption and does not claim exactly-once physical execution. We also characterize four load-bearing assumptions by concrete attacks, give an acceptance-prefix integrity result with an explicit anti-backdating boundary, and show how Ed25519 and ML-DSA context strings instantiate the construction. Four Tamarin models check the corresponding symbolic properties against an attacker that can solicit honest signatures on arbitrary contexts. Nine deliberately weakened variants produce concrete counterexample traces without weakening the signature primitive.

1 Introduction

Per-transaction signatures are an old and useful idea. Payment cards, hardware wallets, transaction-signing authenticators, and authorization tokens all bind some request data to a cryptographic authenticator. The security question studied here begins one layer later: what may an enforcement point infer when the service that assembled the request, displayed it for approval, collected the signature, and retained the resulting record is adversarial?

Suppose an operator can ask an honest approver to sign arbitrarily many valid requests. This is stronger than an attacker that merely steals a bearer token and different from an attacker that extracts the approver’s key. The operator may harvest genuine signatures, replay them, splice signatures from different approval ceremonies, alter unsigned labels, or present the same authorization to concurrent executors. EUF-CMA security rules out a new signature on a message that was never signed. It does not define which application fields must be in that message, whether two signatures refer to the same ceremony, or whether a valid authorization may be admitted twice.

We call the missing protocol property ABIA, short for *Action-Bound Injective Authorization*. “Action-bound” means that every accepted approval signs a closed, injective encoding of the exact action, policy commitment, initiator, audience, authorization instance, validity data, and approver slot. “Injective” means that, within

one enforcement domain, one authorization instance can precede at most one admitted provider attempt. The second property is stateful. A signature cannot make a distributed consume operation linearizable.

The construction uses only standard signatures and hashes. It is not a new signature primitive. Its contribution is a protocol experiment and a reduction that keep three failure classes separate:

1. **Transplantation.** An accepted action was not the context signed by an uncompromised required key.
2. **Duplicate admission.** The same authorization instance precedes two provider-entry events in one enforcement domain.
3. **Quorum substitution.** A 2-of-2 admission succeeds without two distinct pinned approver keys signing the same base authorization, or with the initiator occupying an approver slot.

The main theorem bounds the first and third events by signature unforgeability and hash collision resistance. The second event is impossible only under complete mediation by a linearizable register-and-consume interface. This division is the point of the result. Systems often describe a signed token as “single-use” even though its single-use property exists only in an enforcement service. ABIA names that service and carries its failure probability as a separate term.

The paper contributes:

- a signer-harvesting experiment in which the adversarial operator can obtain valid signatures on adaptively chosen authorization contexts;
- a closed protocol interface for single-approver and 2-of-2 authorization;
- a reduction that separates exact-context authenticity from single-use admission and from complete provider mediation;
- necessity results for injective encoding, consume-if-unused state, initiator binding and exclusion, and fresh non-recreatable registration;
- an acceptance-prefix integrity theorem that survives later key reveal, paired with an impossibility result for offline anti-backdating under ordinary signatures alone;
- symbolic confirmation in Tamarin, including negative models that expose same-authorization replay, stale registry views, downgrade, authority expansion, profile substitution, and action-key substitution when their checks are removed; and
- an explicit boundary: ABIA proves neither human identity nor rendering fidelity, policy correctness, log completeness, or downstream physical effects.

2 Attacks and Necessary Conditions

The following attacks explain why ordinary signature security does not imply ABIA. Every accepted signature in these traces is genuine and verifies. The attacker changes only application encoding, ceremony binding, or mutable state. Four traces are central to the proof, and a fifth shows how the same failure appears in a generic approval workflow.

2.1 Context transplant under non-injective encoding

Suppose distinct typed contexts $x \neq x'$ have the same accepted byte encoding. The operator asks an honest approver to sign x and later presents the returned signature with x' . The verifier reconstructs the same bytes, so the signature verifies even though a security-relevant field changed.

Operator -> Approver:	display x ; request $\text{Sign}(\text{Enc}(x))$
Approver -> Operator:	$\text{sigma} = \text{Sign}(\text{sk}, \text{Enc}(x))$
Operator -> Executor:	present (x', sigma) , $x' \neq x$
Executor:	$\text{Enc}(x') = \text{Enc}(x)$; $\text{Verify} = \text{true}$; accept x'

This is not an EUF-CMA forgery because the accepted message bytes were queried to the signing oracle. It is an application-language failure. A closed, injective encoding prevents the equality above; collision resistance additionally prevents two distinct canonical actions from sharing the action digest embedded in the slot message.

2.2 Replay without admission state

An approver signs message m once and the operator sends the same valid artifact to two executors. Both verify it and enter the provider.

```
Operator -> Approver: request Sign(m)
Approver -> Operator: (m, sigma)
Operator -> Executor1: (m, sigma)      Executor1 -> Provider: entry(u)
Operator -> Executor2: (m, sigma)      Executor2 -> Provider: entry(u)
```

Neither acceptance is a forgery. A nonce inside m is insufficient unless shared state remembers and atomically consumes it. Per-process caches fail when executors race or restart. The 11-step Tamarin counterexample reproduces this trace by removing only the linear `AdmissionAvailable` input.

2.3 Quorum splicing without a shared instance

Suppose two approver slots sign the same action and policy but no common authorization instance. The operator starts ceremonies C_1 and C_2 , obtains a slot-1 signature from C_1 and a slot-2 signature from C_2 , and combines them.

```
Operator -> Approver1: Sign(action, policy, slot1, C1)
Approver1 -> Operator: sigma1
Operator -> Approver2: Sign(action, policy, slot2, C2)
Approver2 -> Operator: sigma2
Operator -> Executor: (sigma1, sigma2) as one alleged quorum
```

Both signatures are genuine, but no two approvers joined one ceremony. A fresh shared instance q , independently issued and included in both slot messages, turns ceremony membership into byte equality checked by the verifier. Per-slot nonces remain distinct and do not replace q .

2.4 Initiator substitution without signed separation of duties

Suppose self-approval is checked against an unsigned initiator label. The operator collects genuine approvals under label i , changes the label to i' , and asks the executor to apply separation of duties to the changed value.

```
Operator -> Approvers: Sign(action, q, slots); unsigned initiator = i
Approvers -> Operator: sigma1, sigma2
Operator:      replace unsigned i by i'
Operator -> Executor: (i', sigma1, sigma2); one approver = i'
Executor:      signatures verify under the substituted label
```

The historical Tamarin model found this attack directly. Binding i into every slot message makes relabeling a signed-message change. The executor must also exclude the signed initiator from every approver slot; binding without exclusion would faithfully authorize self-approval.

2.5 Stale registry-view laundering

An operator obtains valid approvals under an old authority view h_0 . The registry later publishes h_1 , which revokes or narrows that authority. If the executor checks only an operator-presented or unpinned view, the old evidence can be evaluated as though h_0 were still current.

Registry:	publish h0; later publish h1 that revokes authority
Operator -> Approvers:	collect signatures bound to h0
Operator -> Executor:	present signatures plus stale view h0
Executor:	accept because no exact current-head check is required

The composed Tamarin comparison falsifies current-view acceptance in 20 steps. The repair is not to distrust the signatures. It is to bind the relied-upon authority material and require the executor’s independently pinned current view.

2.6 Generic approval-workflow transplant

A common workflow records that a user clicked “approve” and then signs or logs a token containing a request identifier, user identifier, and boolean decision. The consequential action bytes remain in an operator-controlled database row.

Operator -> User:	display action a and an Approve button
User -> Backend:	approve(request_id = R)
Backend -> Operator:	token = Sign(sk, R, user, approved)
Operator:	change database row R from a to a’
Operator -> Executor:	(R, a’, token); signature verifies

This pattern loses the transplantation game immediately: the signed token never commits to the canonical action the executor will submit. No vendor assumption is needed. The minimal repair is to derive the action at the enforcement boundary, place its canonical commitment and the policy, initiator, audience, and fresh authorization instance inside the signed message, and atomically consume the registered admission key.

2.7 Nine machine-found separations

The current artifacts retain nine deliberately weakened comparisons. Each changes one relied-upon check while leaving the ideal signature algebra unchanged.

Attack name	Omitted check	Machine-found consequence
Focused Double Acceptance	checked acceptance consumes state	one genuine receipt is accepted twice in the focused model
Composed Double Entry	provider entry consumes shared state	one genuine authorization reaches provider entry twice after full composition
Stale Head Laundering	exact current registry head	revoked or narrowed authority is accepted under an old view
Class Downgrade	minimum assurance class	weaker evidence is accepted for a stronger class
Denial Becomes Authority	denial non-authorization	a signed denial is treated as positive authority
Scope Expansion	authority-scope pin	an action outside the signed scope is admitted
Profile Swap	complete profile pin	a different verification profile is substituted
Phantom Challenge	registration before exposure	an unregistered challenge is accepted
Action-Key Swap	canonical action-key binding	evidence for one action keys provider entry for another

The 11-step focused replay occurs in Single Approval. The 31-step no-consumption trace and 20-step stale-view trace occur in Composed Reliance. The remaining six paired traces occur in Enforcement Obligations in 14–16 steps. The two replay lemmas expose the same missing state transition at different composition depths, so the artifacts contain nine falsified comparisons but eight attack classes. The initiator-relabeling attack is retained as repaired design history because the strict current quorum model now binds the initiator.

2.8 Necessity results

The same traces give tightness results for the protocol assumptions.

Proposition 1 (Injective encoding is necessary). *If the accepted authorization language admits distinct security contexts $x \neq x'$ with identical signed bytes, then an operator with one honest signature on x wins transplantation for x' with probability one.*

Proof. Query the honest signer on the common byte string and present the returned signature with x' . Verification succeeds on the same bytes. The contexts differ, so transplantation holds, but EUF-CMA is not violated because that byte string was queried. $\square$

Proposition 2 (Consume-if-unused is necessary). *If two provider-entry paths can verify the same authorization without sharing one linearizable consume-if-unused transition, an operator can win duplicate admission using one genuine receipt and no cryptographic break.*

Proof. Deliver the receipt concurrently to the two paths. Both cryptographic checks are identical and succeed. Without shared atomic state, neither path observes that the other admitted the same key. $\square$

Proposition 3 (Signed initiator exclusion is necessary). *If the initiator is not bound into every slot message, an operator can relabel a valid quorum before the separation-of-duties check. If the initiator is bound but not excluded from approver slots, the protocol accepts self-approval directly.*

Proof. The first case is the relabeling trace above. In the second, ask the signed initiator to occupy a required slot and present the resulting valid signature; all checks other than exclusion succeed. $\square$

Proposition 4 (Fresh non-recreatable registration is necessary). *If an operator may register an attacker-chosen stale instance, or recreate UNUSED for a consumed admission key, then it can reuse previously collected signatures to obtain a new successful consume and provider entry.*

Proof. Keep the original signed slots. Reintroduce their shared instance or reset its admission key to UNUSED, then present the unchanged signatures. The verifier reconstructs the original signed messages and the recreated state makes the consume succeed. No forgery or collision is required. $\square$

These attacks are minimal. They require no key extraction or failure of the signature primitive. They differ only in protocol bindings and shared state, which is why those assumptions appear as first-class objects in the definition and theorem.

3 Preliminaries and Assumptions

3.1 Signatures and encoding

Let $\text{Sig} = (\text{Kg}, \text{Sign}, \text{Verify})$ be a digital signature scheme. For each approver j , $(sk_j, pk_j) \leftarrow \text{Kg}(1^\lambda)$. We use the standard EUF-CMA experiment and write $\text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B})$ for the probability that adversary $\mathcal{B}$ produces a valid signature on a message not submitted to its signing oracle.

Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ be collision resistant. Let Enc be a deterministic, injective encoding on a closed typed schema. “Closed” means that a parser rejects unknown members, missing required members, duplicate members, values outside the declared types, and non-canonical representations. The proof needs injectivity, not a particular wire format. A conforming implementation may use a canonical encoding such as deterministic CBOR or JCS over a closed JSON profile, provided it establishes the same property.

Domain-separation labels are literal protocol constants and are included in every encoded tuple. All equality checks below are byte equality after strict parsing; display strings and workflow identifiers are not security inputs.

3.2 Authorization context

A canonical action a contains an action type and every material parameter the executor will use. Let p be a commitment to the exact policy version, i the initiator identifier, r the relying-party audience, and q a fresh shared authorization instance. The base context is

$$b = (a, p, i, r, q).$$

For approver slot v , let n_v be a fresh slot nonce and w_v a closed validity object containing issuance and expiration bounds. The slot context is $x_v = (b, v, n_v, w_v)$ and the signed message is

$$m_{b,v} = \text{Enc}(\text{ABIA-SIGN-v1}, H(\text{Enc}(a)), p, i, r, q, v, n_v, w_v).$$

The shared q binds all slot signatures to one ceremony. Distinct n_v values keep the slot contexts distinct. The relying party or another trusted component for enforcement domain d issues q and durably registers the corresponding admission key

$$u_b = H(\text{Enc}(\text{ABIA-ADMIT-v1}, d, H(\text{Enc}(a)), p, i, r, q)).$$

The adversarial operator may relay q . It cannot cause acceptance under an unregistered q , choose a replacement q , or recreate a consumed registration.

3.3 Linearizable admission service

For each domain d , the admission service exports:

- $\text{RegisterFresh}_d(u)$, which atomically creates state $u \mapsto \text{UNUSED}$ if no state for u has ever existed; and
- $\text{ConsumelfUnused}_d(u)$, which atomically changes $u \mapsto \text{UNUSED}$ to $u \mapsto \text{ADMITTED}$ and returns success, or returns failure without entering a provider for every other state.

Operations are linearizable and durable. No transition recreates UNUSED for the same key. *Complete mediation* means that every entry into a provider in d is preceded by a successful $\text{ConsumelfUnused}_d(u)$ for the same request. These are system assumptions, not consequences of signature security.

4 Protocol

The following algorithms define the analyzed 2-of-2 profile. The single-approver profile uses one slot and the same admission interface.

```

Issue_d(a, p, i, r):
  repeat q <- {0,1}^lambda
  u <- H(Enc("ABIA-ADMIT-v1", d, H(Enc(a)), p, i, r, q))
  until RegisterFresh_d(u) returns success
  return (b = (a,p,i,r,q), u)

Approve(sk_j, b, v_j, n_j, w_j):
  m_j <- Enc("ABIA-SIGN-v1", H(Enc(a)), p, i, r,
    q, v_j, n_j, w_j)
  return (v_j, n_j, w_j, Sign(sk_j, m_j))

VerifySlot(pk_j, b, v_j, n_j, w_j, sigma_j):
  reject unless all inputs parse under the closed schema
  reject unless w_j is current under the relying party's clock policy
  m_j <- Enc("ABIA-SIGN-v1", H(Enc(a)), p, i, r,
    q, v_j, n_j, w_j)
  return Verify(pk_j, m_j, sigma_j)

```

```

VerifyAndConsume_d(b, slots[1], slots[2]):
  reject unless q equals the independently issued instance
  reject unless both slots contain the same base context b
  reject unless ids, pinned keys, slots, and nonces are pairwise distinct
  reject if either approver id equals initiator i
  reject unless VerifySlot succeeds for both pinned keys
  u <- H(Enc("ABIA-ADMIT-v1", d, H(Enc(a)), p, i, r, q))
  reject unless ConsumeIfUnused_d(u) returns success
  emit ProviderEntry(d, u, a)

```

The executor constructs a from the operation it is about to submit. It does not accept an operator-supplied label as the action. The policy commitment p is an exact digest or equivalent immutable identifier, not merely a policy name. The audience r prevents a signature collected for one relying party from being accepted by another relying party that uses different trust and policy inputs.

The protocol emits **ProviderEntry**, not **EffectOccurred**. If the provider accepts a request and its acknowledgement is lost, the outcome is indeterminate. The executor must reconcile using provider state or an idempotency interface. ABIA does not authorize a blind retry.

5 Signature Instantiations and Context Strings

ABIA is signature-scheme agnostic, but two current instantiations clarify the role of cryptographic context strings.

With Ed25519, the approver signs the complete $m_{b,v}$ byte string using the Ed25519 algorithm in RFC 8032 [3]. The literal **ABIA-SIGN-v1** remains inside the injective application encoding. RFC 8032 also defines Ed25519ctx, but deployments need not depend on that variant because many interfaces expose only pure Ed25519. In either case, the verifier reconstructs the complete application message rather than signing an operator-provided digest with an unsigned description beside it.

FIPS 204 standardizes ML-DSA and HashML-DSA and permits a context string of at most 255 bytes [4]. An ML-DSA profile may set that string to the ASCII bytes **ABIA-SIGN-v1** and require the verifier to use the same value. This provides cryptographic domain separation. It does not, by itself, bind the canonical action, policy, initiator, audience, ceremony instance, slot, nonce, or validity object, and it carries no single-use state. Those fields still belong in $m_{b,v}$, and **ConsumeIfUnused** remains necessary.

Thus the standardized context parameter and ABIA solve different problems. The former separates uses of one signature primitive across protocols. The latter defines the full typed statement authorized in this protocol and composes that statement with admission state. Treating an ML-DSA context string as a substitute for the application message would lose the transplantation game while every signature continued to verify.

6 Security Experiments

The challenger generates and pins all approver keys, initializes the admission service for domain d , and gives every public key to adversary $\mathcal{A}$. The adversary controls the network, operator, request transport, signature collector, and artifact presenter. It may:

- adaptively query $\text{Sign}_j(m)$ for any message under any uncompromised approver key j ;
- request issuance of honest authorization instances for chosen (a, p, i, r) tuples;
- replay, suppress, reorder, and splice every returned artifact;
- invoke verification and admission concurrently; and
- reveal designated approver keys, after which no authenticity claim is made for that key.

The signing oracle models signer harvesting. It deliberately does not assume that an honest approver signs only a semantically good request. The theorem asks a narrower question: if a context is accepted under an unrevealed key, was that exact context signed?

6.1 Trace events and corruption timing

The experiment records the following ordered events:

$$\begin{aligned} &\text{Issued}(d, b, u_b), & \text{Signed}(j, x_v, m_{b,v}), \\ &\text{Consumed}(d, u_b), & \text{ProviderEntry}(d, u_b, a). \end{aligned}$$

An issued event occurs only after successful fresh registration of u_b and before any honest signing query for that authorization instance is answered. A consumed event is recorded at the linearization point of a successful consume operation. A provider-entry event is recorded immediately before crossing the provider boundary and only after all cryptographic, policy, and state checks have succeeded.

Key j is unrevealed for an acceptance if no $\text{Reveal}(j)$ event precedes that acceptance. A later reveal does not retroactively erase the trace relation; the theorem is about acceptance-time authenticity. Conversely, once a reveal has occurred, the experiment makes no claim for later signatures under that key. This is the conventional weak-corruption boundary needed by the per-key EUF-CMA reduction. Forward-secure signatures could strengthen the post-compromise story, but they are not assumed here.

The operator controls scheduling. It may interleave issuance, signing queries, verification, and consume calls across arbitrarily many authorizations and may delay a genuine signature until its validity window is nearly exhausted. The challenger gives it no trusted session channel between approvers. Shared ceremony membership therefore comes only from equality of the independently issued q inside each signed context, not from network ordering or an operator-provided session label.

The model treats a provider boundary as an abstract event. Complete mediation requires every such event to be preceded by a successful consume on the same (d, u_b, a) relation. It does not require the provider to complete, acknowledge, or produce the intended downstream effect. Those later events are outside the ABIA experiment.

6.2 Acceptance-prefix integrity and the anti-backdating boundary

We call the following property *acceptance-prefix integrity*. It is the evidentiary forward-integrity statement supported by the present assumptions: a key reveal after an already recorded acceptance does not change which exact context had to be signed before that acceptance.

Lemma 1 (Acceptance-prefix integrity). *Suppose an acceptance under pinned key pk_j occurs at trace position t_a , no reveal of sk_j precedes t_a , and a reveal occurs later at $t_r > t_a$. Except with probability bounded by $\text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_j) + \text{Adv}_H^{\text{cr}}(\mathcal{C}_H)$, a signing-oracle query for the exact accepted slot context occurred before t_a .*

Proof. Stop the experiment at t_a . The later reveal is outside this prefix and cannot affect its distribution. If the accepted message was not queried to the signing oracle before t_a , the pair is an EUF-CMA forgery for j . If it was queried, injective encoding and collision resistance make the queried context equal to the accepted context. The later reveal changes what the adversary can sign after t_r , but it cannot insert a new signing event into the already ordered prefix. $\square$

The Single Approval Tamarin model checks the symbolic counterpart in the named later-reveal lemma: an acceptance before a later RevealLtk has prior user-verification and signing events over the exact accepted context.

Proposition 5 (Offline anti-backdating is impossible here). *Ordinary signatures and a signer-asserted timestamp do not let an offline verifier distinguish a receipt genuinely signed before key reveal from one minted after reveal with an earlier timestamp inside its signed message.*

Proof. After learning sk_j , the adversary runs the honest signing algorithm on a message containing any claimed earlier time. The resulting pair has the same verification relation as a pair produced before compromise. An offline verifier given only (m, σ, pk_j) has no independent event ordering with which to separate the two cases. $\square$

A stronger anti-backdating claim needs an independently witnessed acceptance before the reveal, such as inclusion in an append-only signed checkpoint with trusted ordering, or a forward-secure signature scheme whose erased epoch keys cannot be reconstructed after compromise. Neither mechanism is assumed in ABIA, so the paper makes the prefix claim and not the stronger offline claim.

Definition 1 (Transplantation). *$\mathcal{A}$ wins Transplant if the verifier accepts $x^* = (b^*, v^*, n^*, w^*)$ under an unrevealed pinned key, but the signing oracle for that key was never queried on m_{b^*, v^*} .*

Definition 2 (Duplicate admission). *$\mathcal{A}$ wins Duplicate if two distinct provider-entry events in the same domain d are both labeled by the same admission key u_b .*

Definition 3 (Quorum substitution). *$\mathcal{A}$ wins QuorumSub if a 2-of-2 provider entry occurs while at least one of the following is true: fewer than two distinct pinned approver identifiers and keys were accepted; a required unrevealed key did not sign its accepted slot message; the two slots do not share byte-identical b ; or the signed initiator occupies an approver slot.*

The combined experiment ABIA returns one if any of the three events occurs. Let **AtomicityFail** $_d$ denote any violation of the register-and-consume contract, and let **MediationBypass** $_d$ denote a provider entry that did not pass through that contract.

7 Security Argument

Lemma 2 (Exact-context binding). *Conditioned on no collision in H and injectivity of **Enc**, equality of two valid slot messages implies equality of their action, policy commitment, initiator, audience, authorization instance, slot, slot nonce, and validity object.*

Proof. The outer encodings carry the same domain-separation label and parse under one closed schema. Injectivity therefore gives equality of every encoded field. In particular, the action digests are equal. If the canonical action encodings were different, they would form a collision in H ; conditioned on no such collision, the actions are equal. All remaining fields occur directly in the injective outer encoding and are therefore equal. $\square$

Lemma 3 (Transplantation reduction). *For every transplantation adversary $\mathcal{A}$, there are EUF-CMA forgers $\mathcal{B}_j$ and a collision finder $\mathcal{C}_H$ such that*

$$\Pr[\text{Transplant}_{\mathcal{A}} = 1] \leq \sum_j \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_j) + \text{Adv}_H^{\text{cr}}(\mathcal{C}_H).$$

Proof. The reduction is made explicit in the game sequence below. In short, an accepted signature under unrevealed key pk_j either has no matching signing-oracle query, in which case it is an EUF-CMA forgery for j , or has a matching query on the same bytes. In the latter case the exact-context-binding lemma makes the accepted context equal to the signed context unless a hash collision occurred. A union bound over the pinned required keys and the collision event gives the inequality. $\square$

Lemma 4 (Injective admission). *Conditioned on linearizability, durable state, and complete mediation in domain d , $\Pr[\text{Duplicate}_{\mathcal{A}} = 1] = 0$.*

Proof. By complete mediation, both provider entries must follow successful **ConsumeIfUnused** $_d(u_b)$ operations. Linearizability places those calls in a total order consistent with real time. The first success changes the only state for u_b from UNUSED to ADMITTED. No allowed transition recreates UNUSED, so the second call cannot also succeed. A duplicate entry thus implies either a state-service failure or a path around the service. $\square$

7.1 Game sequence

We now give the hybrids underlying the concrete bound. Each game returns one if the real ABIA experiment returns one, except where the game explicitly aborts. The challenger records every signing query as a pair (j, m) and every issued authorization as (d, a, p, i, r, q, u_b) . It also records the linearization point and return value of every register and consume operation. This bookkeeping is not visible to the adversary.

Game G_0 : the real experiment. The challenger runs the protocol and adversary exactly as defined in Section 6. Thus

$$\Pr[G_0 = 1] = \Pr[\text{ABIA}_{\mathcal{A}} = 1].$$

Game G_1 : exclude action-hash collisions. The challenger aborts if two distinct canonical action encodings queried or accepted during the experiment have the same hash. An algorithm $\mathcal{C}_H$ that runs G_0 and outputs the first such pair is a collision finder. Therefore

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \text{Adv}_H^{\text{cf}}(\mathcal{C}_H).$$

Conditioned on no abort, equality of action digests implies equality of canonical action encodings. Together with injectivity of Enc , equality of signed-message bytes implies equality of every field in the slot context.

Game G_2 : exclude accepted unqueried messages. For each unrevealed pinned key j , the challenger aborts if verification accepts (m, σ) under pk_j while (j, m) is absent from the signing-query table. The reduction for one slot is the following explicit algorithm.

```

Forger B_j(pk*, Sign*):
  set pk_j <- pk*; generate (sk_k, pk_k) for every k != j
  initialize the ABIA experiment and run A on all public inputs
  on A.Sign(j, m):      record (j,m); return Sign*(m)
  on A.Sign(k, m), k!=j: record (k,m); return Sign(sk_k,m)
  on A.Reveal(k), k!=j: return sk_k
  on A.Reveal(j):      abort this fixed-j reduction
  on accepted (m,sigma) under pk_j before Reveal(j):
    if (j,m) is absent from the query table, output (m,sigma)
    otherwise continue the simulation

```

Registration, consumption, scheduling, and all public computations are simulated exactly. The fixed- j simulation is perfect up to either the first reveal of j or the accepted-unqueried event. A reveal before that event removes j from the experiment's authenticity claim. If the event occurs first, $\mathcal{B}_j$ stops immediately, so it never needs to reveal the EUF-CMA challenge secret. The message was not queried to its oracle and the signature verifies, which is a valid EUF-CMA forgery.

The standard multi-user union bound gives

$$|\Pr[G_1 = 1] - \Pr[G_2 = 1]| \leq \sum_j \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_j).$$

The sum includes only unrevealed keys required by the accepted policy. A key reveal removes that key from the authenticity claim instead of asking the reduction to simulate a forbidden reveal of its challenge secret.

Game G_3 : replace accepted slots by recorded exact contexts. In G_2 , every accepted slot under an unrevealed key has a matching signing query on exactly the accepted message bytes. The challenger now associates that slot with the corresponding recorded query. This is a conceptual rewrite and changes no probability. By the closed-schema checks, injectivity of Enc , and the no-collision condition from G_1 , the recorded query and accepted slot agree on the action, policy commitment, initiator, audience, authorization instance, slot, slot nonce, and validity object. Transplantation is therefore impossible in G_3 .

Game G_4 : condition on the admission contract. The challenger aborts on AtomicityFail_d or MediationBypass_d . This changes the success probability by at most

$$\Pr[\text{AtomicityFail}_d \vee \text{MediationBypass}_d].$$

In G_4 , every provider entry follows a successful consume operation. If two entries carried the same u_b , their successful consumes would be ordered by linearizability. The first moves the unique durable state for u_b from UNUSED to ADMITTED; the second cannot succeed. Duplicate admission is therefore impossible.

The verifier’s deterministic distinctness, shared-base-context, pinned-key, and initiator-exclusion checks remain. In G_3 every accepted unrevealed slot is an exact recorded signing query, so those checks also make quorum substitution impossible. Hence $\Pr[G_4 = 1] = 0$.

Table 1: Advantage accounting for the game sequence.

Transition	Bad event	Reduction or assumption	Probability charge
$G_0 \rightarrow G_1$	distinct canonical actions share one digest	collision finder $\mathcal{C}_H$	$\text{Adv}_H^{\text{cr}}(\mathcal{C}_H)$
$G_1 \rightarrow G_2$	an unrevealed required key accepts an unqueried message	one explicit forger $\mathcal{B}_j$ per required key	$\sum_j \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_j)$
$G_2 \rightarrow G_3$	replace accepted slots by their recorded equal-byte queries	injectivity and the G_1 condition	0
$G_3 \rightarrow G_4$	atomic state fails or provider entry bypasses mediation	deployment contract	$\Pr[\text{AtomicityFail}_d \vee \text{MediationBypass}_d]$
G_4	any ABIA winning condition remains	deterministic verifier checks	0

Each $\mathcal{B}_j$ makes exactly the signing-oracle queries that $\mathcal{A}$ makes for key j and adds only key generation for the other slots, protocol simulation, and table lookups. There is no guessing loss beyond the displayed multi-user sum because the analysis defines one reduction per required key and applies a union bound.

7.2 Simulation and state boundaries

The signature reduction does not simulate the admission service cryptographically. Register and consume are ordinary oracle calls in the experiment, and their failure probability remains an explicit term. This is intentional. Replacing that term with a negligible cryptographic advantage would silently assume the conclusion that concurrent single-use admission is correct.

The reduction also does not treat expiration, policy selection, identity proofing, or display fidelity as signature properties. The verifier checks the closed validity object and pinned policy inputs before acceptance, but the correctness of those inputs belongs to the relying party. Likewise, a signature over action a does not prove that a human was shown a faithful rendering of a . The experiment only establishes which bytes were signed and which action the executor admitted.

Lemma 5 (Quorum binding). *Conditioned on no signature forgery or hash collision, the 2-of-2 verification algorithm cannot satisfy QuorumSub for unrevealed required keys.*

Proof. The algorithm checks identifier, key, slot, and nonce distinctness and rejects the signed initiator in either approver slot. It checks byte equality of the base context before verifying one signature for each pinned slot key. If a required slot message was not queried to that key’s signing oracle, its accepted signature is a forgery. If both queries exist, exact-context binding makes both signed slot contexts equal to the accepted contexts. The explicit base-context check then makes both signatures authorize the same action, policy, initiator, audience, and authorization instance. The explicit distinctness and initiator-exclusion checks rule out the remaining winning conditions. $\square$

Theorem 1 (ABIA composition). *Let Sig be EUF-CMA secure, H collision resistant, and Enc injective on the closed authorization schema. Consider any probabilistic polynomial-time adversary $\mathcal{A}$ in the signer-harvesting experiment. There are forgers $\mathcal{B}_j$ and a collision finder $\mathcal{C}_H$ such that*

$$\Pr[\text{ABIA}_{\mathcal{A}} = 1] \leq \sum_j \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_j) + \text{Adv}_H^{\text{cr}}(\mathcal{C}_H) + \Pr[\text{AtomicityFail}_d \vee \text{MediationBypass}_d].$$

For the 2-of-2 profile, the sum ranges over the two unrevealed pinned approver keys required by the accepted policy.

Proof. Games G_0 and G_1 differ only on an action-hash collision. Games G_1 and G_2 differ only on an accepted signature under an unrevealed pinned key whose message was not queried to that key’s signing oracle. The reductions above bound those differences by the collision and per-key EUF-CMA advantages. Game G_3 is identical to G_2 but uses exact-context binding to name the matching signing queries, so transplantation and the signature-dependent quorum conditions cannot occur. Game G_4 additionally conditions on successful atomic admission and complete mediation, which excludes duplicate admission. The verifier’s remaining quorum conditions are deterministic checks. Thus G_4 has zero success probability, and summing the game differences proves the inequality. $\square$

Theorem 2 (2-of-2 quorum corollary). *Suppose the 2-of-2 verifier admits base authorization b with slot records x_1, x_2 under distinct pinned keys pk_1, pk_2 , neither key was revealed, and neither approver identifier equals the signed initiator. Except with probability bounded by*

$$\text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_1) + \text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}_2) + \text{Adv}_H^{\text{cr}}(\mathcal{C}_H),$$

both pinned keys previously signed their assigned slot contexts over the same action, policy commitment, initiator, audience, and authorization instance. No single key or signed initiator fills both slots.

Proof. Apply Games G_1 through G_3 to the two required slots. In G_2 , an accepted slot without a matching oracle query is a forgery under that slot’s key. In G_3 , each matching query binds the exact accepted slot fields. The verifier checks byte equality of b , distinct slot identifiers, distinct approver identifiers, distinct pinned keys, distinct nonces, and exclusion of the signed initiator. Those deterministic checks give the stated 2-of-2 property. The admission-service term is absent because this corollary concerns quorum authenticity, not single-use provider entry. $\square$

7.3 Concrete loss and interpretation

Let N be the number of unrevealed pinned keys that may be required by an accepted policy. If every per-key EUF-CMA advantage is at most ϵ_{sig} , the collision advantage is at most ϵ_H , and the measured or assumed probability of either admission failure event is at most ϵ_{state} , the theorem gives

$$\Pr[\text{ABIA}_{\mathcal{A}} = 1] \leq N\epsilon_{\text{sig}} + \epsilon_H + \epsilon_{\text{state}}.$$

The signature reduction preserves the adversary’s signing-query count for the embedded key and adds only the cost of protocol simulation and table lookups. There is no hidden claim that ϵ_{state} is negligible. It is a deployment term whose justification must come from the implementation of the shared admission domain, including its concurrency, durability, and bypass controls.

The bound is deliberately conservative in two ways. First, it uses a union bound over required keys even when policy structure may make simultaneous failures necessary. Second, it charges any detected action-hash collision once, regardless of whether the collision would satisfy all remaining acceptance checks. Those choices keep the reduction independent of one signature algorithm and one policy cardinality. The price is a simple bound rather than a tight system-specific one.

The theorem gives at most one admitted provider attempt per authorization inside d . It does not establish that the provider performed a physical effect, that an effect occurred exactly once, or that a second independent domain will reject the same authorization. Cross-domain single use requires a shared linearizable admission service or an equivalent coordination protocol.

8 Symbolic Analysis

The computational proof isolates the assumptions. Four Tamarin models check the same bindings under a Dolev-Yao network attacker. The models use Tamarin’s signing theory, so verification follows from the signing equations rather than from an axiom saying that a signature is valid. The adversary controls the network, learns all public keys, may reveal a device key through an explicit rule, and may ask an honest approver to sign arbitrary attacker-chosen contexts.

User verification is a linear precondition to signing. Admission availability is also a linear fact and checked acceptance consumes it. No trace restriction states that acceptance is unique. This use of linear state follows the security-API tradition: mutable authorization state is part of the model, not an informal note outside it.

8.1 Four models and checked obligations

The filenames remain available with the artifact, but we use descriptive names in the paper so that the result does not depend on project terminology.

Model	Question	Strict result	Unsafe comparison
Single Approval	Does acceptance imply a prior user-verified signature over the exact context, remain bound before a later reveal, and is checked acceptance injective?	1 witness and 4 all-traces lemmas verified	replay without consumption, falsified in 11 steps
Quorum	Does 2-of-2 require two distinct pinned signers over the same action and ceremony, excluding the initiator?	1 witness and 4 all-traces lemmas verified	historical initiator-relabeling trace when initiator binding was absent
Composed Reliance	Do exact action, profile, audience, authority view, issuer, quorum, and consumption remain bound through provider entry?	1 witness and 9 all-traces lemmas verified	replay in 31 steps; stale registry view in 20 steps
Enforcement Obligations	Do downgrade refusal, denial non-authorization, scope and profile pinning, challenge consumption, and action-keyed entry hold together?	2 witnesses and 8 all-traces lemmas verified	6 weakened lemmas falsified in 14–16 steps

Across the four models, 25 all-traces obligations and five executable witnesses verify. Nine deliberately weakened current-model comparisons produce attack traces. The earlier quorum falsification is retained as design history but is not counted among those nine current-model comparisons.

8.2 Symbolic-to-computational correspondence

The symbolic and computational analyses are not independent feature lists. The table identifies which Tamarin state or event realizes each object in the games.

Tamarin object	Computational object	Shared security meaning
Human_Sign_Receipt, Signed	$\text{Sign}_j(m)$ query and query-table row	the attacker may harvest genuine signatures on chosen contexts
$\text{Eq}(\text{verify}(\dots), \text{true})$ under PinnedKey	$\text{Verify}(pk_j, m, \sigma)$ under a pinned slot key	accepted bytes verify for the required key
UV before Signed	approver-side signing precondition	user verification gates signing, but does not prove rendering fidelity
RevealLtk ordered before or after Accept	weak-corruption boundary at acceptance	later reveal does not alter the accepted prefix; earlier reveal removes authenticity
AdmissionAvailable as a linear fact	successful $\text{RegisterFresh}_d(u)$ with state UNUSED	one fresh authorization instance has one consumable admission state
AcceptChecked consuming that fact	successful $\text{ConsumelfUnused}_d(u)$ before provider entry	injectivity comes from mutable state, not signature algebra
Core authenticity and cross-action lemmas	Games G_1 – G_3	exact accepted context has a prior matching signing query
Later-reveal prefix lemma	acceptance-prefix lemma	a later key reveal does not rewrite the already recorded trace prefix
Injective checked-acceptance lemma	Game G_4	one registered key precedes at most one checked admission
Unchecked-injectivity comparison	Duplicate without the state contract	removing consumption yields a concrete replay
Quorum commit and distinct-slot lemmas	QuorumSub and its corollary	two distinct pinned keys sign one byte-identical base authorization, excluding the initiator

The correspondence has one deliberate asymmetry. Injectivity of the application encoding and computational hardness of collisions are assumptions in the symbolic model’s term algebra, while Games G_0 – G_3 expose and charge them explicitly. Conversely, Tamarin explores arbitrary symbolic interleavings, while the computational theorem carries state failure and mediation bypass as external probability terms.

8.3 Single approval and quorum

In Single Approval, the request rule creates one **AdmissionAvailable** fact for a fresh context. Checked acceptance consumes it. The exact tool results are:

```

executable_honest_receipt (exists-trace): verified (9 steps)
core_authenticity_uv_gated (all-traces): verified (11 steps)
acceptance_prefix_integrity_after_later_reveal (all-traces): verified (12 steps)
no_replay_across_actions (all-traces): verified (11 steps)
injective_acceptance_with_consumption (all-traces): verified (11 steps)
unchecked_acceptance_is_injective (all-traces): falsified - found trace (11 steps)

```

The unsafe rule changes only one thing: it accepts without consuming the linear fact. The attacker delivers one genuine signature twice. No signature forgery or key reveal occurs. This is the symbolic counterpart of the admission-service term in Theorem 1.

Quorum enrolls two distinct approver identities under two pinned keys. Each signature binds the action, initiator, shared authorization instance, slot, and nonce. Commitment requires both signatures over the same base context and excludes the initiator. Its five current lemmas verify in 4–27 steps:

```

executable_quorum (exists-trace): verified (13 steps)
quorum_requires_two_distinct_uv_gated_signatures (all-traces): verified (27 steps)
initiator_cannot_self_approve (all-traces): verified (4 steps)
no_single_signer_fills_quorum (all-traces): verified (4 steps)
commit_requires_signature_over_that_action (all-traces): verified (8 steps)

```

An earlier revision omitted the initiator from the signed tuple. Tamarin then found a trace in which two honest signatures collected under one initiator label were committed under another. Adding the initiator to the signed context restored the original lemmas; no lemma was weakened.

8.4 Composed models

Composed Reliance places challenge issuance, exact action identity, profile and audience, initiator, two distinct approvals, scoped authority, an exact registry view, revocation status, issuer pinning, consumption, and provider entry in one trace. Its strict path includes the following results:

```
execution_requires_full_composition: verified (101 steps)
no_issuer_laundering: verified (785 steps)
no_cross_action_profile_or_audience_replay: verified (41 steps)
execution_has_honest_approvals_or_prior_compromise: verified (178 steps)
injective_execution_with_consumption: verified (250 steps)
```

Seven additional sanity or binding lemmas also verify. Removing consumption produces a 31-step duplicate-entry trace. Removing exact registry-head binding produces a 20-step stale-view trace.

Enforcement Obligations isolates six relied-upon checks and two sanity properties. The strict model verifies downgrade refusal, signed-denial non-authorization, authority-scope pinning, complete profile pinning, registration-before-exposure and one-time challenge use, canonical action-keyed provider entry, and fail-closed reservation. Six paired rules each omit one check. Tamarin finds the corresponding downgrade, denial-as-authorization, authority expansion, profile substitution, unregistered-challenge, and action-key substitution traces in 14–16 steps.

These composed models are confirmation and separation tests, not additional cryptographic theorems. They show that the declared bindings survive symbolic composition and that removing each load-bearing check admits an attack while the signature algebra stays ideal.

8.5 Mechanized-analysis boundary

The four models abstract WebAuthn internals, canonical parsers, natural-person identity, trusted rendering, clocks, database durability, directory publication, transparency-log completeness, and provider effects. A symbolic exact registry view does not prove how that view was published. A linear fact models an atomic state transition but does not prove a particular database implementation. The models establish properties of the specified protocol under these declared abstractions, not security of a deployment.

9 Relation to Prior Work

FIDO2 and WebAuthn security. Barbosa, Boldyreva, Chen, and Warinschi give a modular computational analysis of WebAuthn and CTAP2 and prove authentication properties under their stated client, token, and corruption models [5]. Guan et al. model WebAuthn and CTAP2 together in ProVerif, while Schrempf gives a Tamarin model with an explicit human role [6, 7]. Those works analyze authentication, credential use, and the authenticator-client-relying-party ceremony. ABIA starts after that interface: it assumes an enrolled signing key and asks what an executor may infer when the operator that chooses, renders, collects, stores, and presents action approvals is adversarial. Prior FIDO2 ceremony results do not define single-use provider admission for an action authorization. The flat delta is: ABIA makes the operator the signature-harvesting adversary and separates exact signed-action authenticity from stateful single-use admission.

Transaction authorization. Work on smart-card payment protocols already distinguishes cryptographic proof of critical transaction data from a reusable authentication event and observes that publicly verifiable signatures provide different dispute evidence than shared-key MACs [10]. Hardware-wallet designs such as Notary focus on trusted transaction display and correct device behavior [11]. The earlier “see what you sign” line makes trustworthy input and output part of binding electronic signatures to human intent [12]. ABIA assumes no solution to that display problem. Its narrower contribution is the collector-adversarial experiment and the separation between exact signed context, ceremony binding, and stateful single-use admission after signatures have been collected.

Security APIs and mutable protocol state. Formal analyses of PKCS#11 show why mutable global state belongs inside a security model: API safety can change when key attributes and permitted operations change over time [8]. ABIA uses the same broad methodological lesson, although for a different goal. The linear admission fact is not a cryptographic primitive. It represents the state transition that makes an otherwise replayable valid authorization single-use. The positive and negative Tamarin models keep the signature theory fixed and change only that state input.

Policy-aware signatures. Policy-Compliant Signatures make policy satisfaction part of an enhanced signature primitive and provide privacy and composable-security notions for attribute policies [13]. Attribute-based signatures similarly hide the signer’s identity behind an attribute predicate [14]. ABIA provides neither attribute privacy nor a new signature primitive. It uses ordinary signatures under explicitly pinned keys and studies replay, ceremony splicing, initiator substitution, and single-use admission by an external state service. These are orthogonal properties. A policy-compliant or attribute-based signature could replace the ordinary slot signature, but the admission theorem would still require the state-service assumption.

Capabilities and least privilege. SPKI/SDSI, PolicyMaker, and KeyNote established public-key authorization and trust management for distributed systems [15, 16]. Macaroons add contextual caveats and decentralized attenuation to bearer credentials [17]. StatefulAuth allows client-defined authorization policies to depend on protected request history [18]. These systems primarily decide what a principal or bearer token may do. ABIA asks a different evidence question: whether an accepted action is exactly the action signed by required approver keys under a collector that may harvest honest signatures, and whether one such authorization preceded at most one provider entry. StatefulAuth is especially close in recognizing that authorization properties can require protected state. ABIA makes the stronger atomicity and complete-mediation assumptions explicit because a history integrity tag alone does not serialize concurrent consumption.

Formal protocol analysis. Tamarin and related symbolic tools have been used extensively for authentication and key-exchange protocols [9, 2]. Our use of Tamarin is conventional. The contribution is the adversary query pattern and the paired positive and negative models: the attacker may collect signatures on arbitrary contexts, and the same ideal signing theory is used on both sides of each separation result.

10 Limitations

ABIA proves that an unrevealed enrolled key signed an exact context. It does not prove that the key belongs to a particular natural person. That claim depends on directory governance and identity proofing. Pairwise-distinct enrolled identifiers also do not prove two independent human wills.

The operator may render action X while causing the signing device to sign action Y . ABIA makes Y recoverable and prevents the resulting signature from being used for Z , but it cannot prove what the approver perceived. High-consequence deployments need a trusted or independently authored display path. This is a separate assumption, not a consequence of canonical encoding.

The theorem assumes the executor constructs the action from the operation it will submit and that every provider entry passes through the admission service. An operator able to bypass that executor is outside the enforcement claim and appears as MediationBypass_d in the bound.

The proof covers one domain and the stated single-approver and 2-of-2 profiles. It does not cover arbitrary threshold policies, key-directory publication, revocation history, transparency-log consistency, clock synchronization, or provider-side effects. A lost provider acknowledgement produces an indeterminate outcome. Retrying safely requires provider reconciliation or a provider-enforced idempotency key.

11 Conclusion

A signed authorization has two different security obligations. The signature and closed encoding bind the authorization to an exact context. A linearizable, completely mediated state transition binds that authoriza-

tion to at most one admitted provider attempt. Neither obligation implies the other. ABIA formalizes their composition against an operator that may harvest genuine signatures on arbitrary contexts. Its proof reduces context substitution to standard cryptographic failures and leaves duplicate admission honestly attached to the state service that enforces it. The Tamarin counterexamples show the same boundary operationally: ideal signatures do not stop replay when consumption is removed, and they do not stop initiator relabeling when the initiator is not signed.

Artifact Availability

The four Tamarin theories, replay traces, exact tool summaries, and pinned rerun scripts are available in the public repository: <https://github.com/emiliaprotocol/emilia-protocol>. The theory files are:

```
formal/tamarin/ep_receipt_core.spthy
formal/tamarin/ep_quorum_core.spthy
formal/tamarin/ep_reliance_composed.spthy
formal/tamarin/ep_six_claim_composed.spthy
```

The protocol definition, games, and proofs in this paper are self-contained; no implementation or Internet-Draft is required to evaluate the result.

References

- [1] S. Goldwasser, S. Micali, and R. Rivest, “A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks,” *SIAM Journal on Computing*, 17(2), 1988.
- [2] D. Dolev and A. C. Yao, “On the Security of Public Key Protocols,” *IEEE Transactions on Information Theory*, 29(2), 1983.
- [3] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” RFC 8032, January 2017. <https://www.rfc-editor.org/rfc/rfc8032>.
- [4] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, August 2024. <https://doi.org/10.6028/NIST.FIPS.204>.
- [5] M. Barbosa, A. Boldyreva, S. Chen, and B. Warinschi, “Provable Security Analysis of FIDO2,” *CRYPTO 2021, Part III*, LNCS 12827, pp. 125–156, 2021. https://doi.org/10.1007/978-3-030-84252-9_5.
- [6] J. Guan, H. Li, H. Ye, and Z. Zhao, “A Formal Analysis of the FIDO2 Protocols,” *ESORICS 2022*, LNCS 13556, pp. 3–21, 2022. https://doi.org/10.1007/978-3-031-17143-7_1.
- [7] L. Schrempf, “Formal Verification of FIDO2 with Human Interaction,” Master’s thesis, ETH Zurich, 2023. <https://doi.org/10.3929/ethz-b-000610315>.
- [8] S. Delaune, S. Kremer, and G. Steel, “Formal Security Analysis of PKCS#11 and Proprietary Extensions,” *Journal of Computer Security*, 18(6), pp. 1211–1245, 2010. <https://doi.org/10.3233/JCS-2009-0394>.
- [9] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” *CAV 2013*, LNCS 8044, pp. 696–701, 2013.
- [10] E. Van Herreweghen and N. Ferguson, “Security Mechanisms in EMV,” *USENIX Workshop on Smart-card Technology*, 1999.
- [11] A. Athalye, A. Belay, M. F. Kaashoek, R. Morris, and N. Zeldovich, “Notary: A Device for Secure Transaction Approval,” *27th ACM Symposium on Operating Systems Principles*, pp. 97–113, 2019. <https://doi.org/10.1145/3341301.3359661>.

- [12] A. Weber, “See What You Sign: Secure Implementations of Digital Signatures,” *5th International Conference on Intelligence and Services in Networks*, LNCS 1430, pp. 509–520, 1998. <https://doi.org/10.1007/BFb0056995>.
- [13] C. Badertscher, C. Matt, and H. Waldner, “Policy-Compliant Signatures,” *TCC 2021*, LNCS 13043, pp. 299–329, 2021. <https://eprint.iacr.org/2021/1234>.
- [14] H. Maji, M. Prabhakaran, and M. Rosulek, “Attribute-Based Signatures: Achieving Attribute-Privacy and Collusion-Resistance,” *Cryptology ePrint Archive*, Paper 2008/328, 2008.
- [15] C. M. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen, “SPKI Certificate Theory,” RFC 2693, 1999.
- [16] M. Blaze, J. Feigenbaum, and A. D. Keromytis, “KeyNote: Trust Management for Public-Key Infrastructures,” RFC 2704, 1999.
- [17] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrabie, and M. Lentzner, “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” *NDSS*, 2014.
- [18] L. Cao, L. Meng, D. Stefan, and E. Fernandes, “Stateful Least Privilege Authorization for the Cloud,” *33rd USENIX Security Symposium*, pp. 3477–3495, 2024.